Resenha: Object Constraint Language (OCL): a Definitive Guide (Cabot e Gogolla)

OCL como complemento textual para especificar modelos UML com precisão e apoiar engenharia dirigida por modelos

Referência

CABOT, Jordi; GOGOLLA, Martin. **Object Constraint Language (OCL): A Definitive Guide**. In: BERNARDO, Marco; CORTELLESSA, Vittorio; PIERANTONIO, Alfonso (eds.). *Formal Methods for Model-Driven Engineering*. Berlin, Heidelberg: Springer, 2012. LNCS, v. 7320, p. 58-90. DOI: 10.1007/978-3-642-30982-3_3.

Fonte 1 | Fonte 2. Referências conferidas em 10 out. 2026.

1. Introdução

O texto de Jordi Cabot e Martin Gogolla apresenta a Object Constraint Language como uma resposta a uma limitação recorrente dos modelos gráficos: diagramas UML são úteis para representar estrutura, classes, associações e multiplicidades, mas não conseguem expressar todos os detalhes semânticos de um domínio. A tese central do capítulo é que OCL complementa esses modelos com uma linguagem textual, tipada, declarativa e livre de efeitos colaterais, capaz de registrar invariantes, consultas, regras derivadas e contratos de operações. Assim, o modelo deixa de ser apenas um desenho compreensível visualmente e passa a carregar restrições verificáveis sobre o comportamento esperado do sistema.

2. Síntese das ideias principais

O capítulo parte da motivação para usar OCL junto a UML, apresenta os principais tipos de expressão, detalha aspectos da linguagem e discute suporte ferramental, validação, geração de código e desafios de pesquisa. A leitura mostra OCL como uma ponte entre modelagem conceitual, especificação formal e engenharia dirigida por modelos.

2.1 Limites dos diagramas gráficos

O primeiro argumento do texto é que linguagens gráficas precisam limitar sua expressividade para permanecerem legíveis. Um diagrama de classes pode mostrar que clientes, carros, reservas e filiais estão relacionados, mas não responde sozinho a perguntas como: clientes bloqueados podem alugar carros, uma licença precisa valer durante todo o aluguel, um carro já reservado pode ser escolhido novamente ou como o preço é calculado. OCL entra exatamente nesse espaço: ela registra regras de negócio e restrições que seriam ambíguas, longas ou invisíveis se ficassem apenas em anotações informais ao lado do diagrama.

2.2 Tipos de expressão em OCL

Os autores mostram que OCL não serve apenas para escrever invariantes. A linguagem também pode definir valores iniciais de propriedades, regras de derivação, operações de consulta e contratos com precondições e pós-condições. Isso amplia seu uso porque diferentes aspectos do modelo podem ser especificados com o mesmo vocabulário. Um invariante verifica se todo objeto respeita uma condição; uma regra derivada explica como um valor deve ser calculado; e uma pós-condição descreve o estado esperado após uma operação. O resultado é uma documentação mais precisa e mais próxima da lógica real do sistema.

2.3 Sistema de tipos, coleções e navegação

Uma parte relevante do capítulo é dedicada ao sistema de tipos de OCL. A linguagem trabalha com tipos básicos, classes do modelo, enumerações, tuplas e coleções como Set, Bag, Sequence e OrderedSet. Essa diferenciação é importante porque ordem e repetição alteram o significado de uma expressão. O texto também explica a navegação por associações, a distinção entre null e invalid, além dos valores booleanos, elementos que tornam a linguagem mais expressiva, mas também mais exigente. Na prática, escrever boas expressões OCL demanda entender tanto o modelo UML quanto as regras formais de avaliação da linguagem.

2.4 Ferramentas, validação e geração de código

O capítulo apresenta um panorama de ferramentas para parsing, integração com UML, validação e verificação. Ambientes como MDT/OCL, Dresden OCL, Papyrus, MagicDraw e USE mostram que OCL pode ser analisada, executada contra estados de objetos e usada para detectar modelos super-restritos, sub-restritos ou inconsistentes. Os autores também discutem geração de código a partir de restrições, seja em triggers de banco de dados, seja em verificações dentro de métodos. Esse ponto é prático: uma restrição escrita no modelo só produz valor completo quando pode orientar testes, validações ou código executável.

2.5 Limites e desafios da linguagem

Apesar de sua utilidade, OCL não é apresentada como solução perfeita. A linguagem é expressiva, mas pode ficar verbosa, difícil para iniciantes e dependente de ferramentas que nem sempre implementam a especificação de forma uniforme. Os autores também apontam desafios como modularização da biblioteca padrão, melhorias sintáticas, tratamento de indefinição, escalabilidade da análise e formação de uma comunidade mais forte. Isso mostra que a adoção de OCL exige equilíbrio: ela aumenta precisão, mas introduz custo de aprendizado e necessidade de suporte ferramental adequado.

3. Aplicação imaginada dos conceitos

Em um sistema de locação de veículos, o diagrama de classes poderia mostrar entidades como Cliente, Veiculo, Reserva, Filial e Pagamento. Mesmo assim, regras importantes ficariam ausentes se dependessem apenas das caixas e associações do UML. Com OCL, seria possível registrar invariantes como: um veículo não pode estar em duas reservas ativas no mesmo período; um cliente bloqueado não pode abrir nova reserva; a data final deve ser posterior à data inicial; e a carteira do motorista deve permanecer válida até o fim da locação. Essas expressões deixariam explícitas decisões que, de outra forma, poderiam ficar espalhadas em reuniões, comentários ou validações duplicadas no código.

Em um projeto real, OCL poderia funcionar como camada de especificação entre análise e implementação. O time modelaria as classes principais, escreveria restrições para as regras críticas e usaria essas expressões como base para testes automatizados, validações de entrada e documentação técnica. Mesmo que o código final seja feito em Java, as regras OCL ajudariam a manter rastreabilidade entre modelo, domínio e implementação. O cuidado é não tentar especificar tudo: o melhor uso está nas restrições de negócio mais importantes, especialmente aquelas que afetam consistência, integridade e contratos de operações.

4. Conclusão

A principal contribuição do texto é mostrar que modelos visuais precisam de uma linguagem complementar para alcançar precisão sem perder clareza. OCL ocupa esse papel ao permitir que regras formais sejam associadas a modelos UML, tornando explícitas restrições, derivações, consultas e contratos. O capítulo também é valioso por não limitar a discussão à sintaxe: ele relaciona a linguagem a ferramentas, validação, geração de código e desafios de adoção. Como aprendizado, fica a ideia de que OCL é mais útil quando usada de forma seletiva e disciplinada, documentando regras críticas que precisam ser compreendidas, verificadas e preservadas durante a evolução do software.

Nota de revisão técnica

A distinção entre null e invalid foi conferida também na especificação OCL 2.4 da OMG, seções 7.4.10, 11.3.2 e 11.3.3. A versão e a ferramenta precisam ser verificadas ao aplicar as restrições.